

Night Market

After a long day of APIO training, Alice and Bob plan to go to a night market for dinner. Since they are craving cookies, they decided to head to a night market where every stand sells cookies shaped like digits.

The night market is structured like an $N \times M$ grid, where rows are indexed $0, 1, \dots, N - 1$ from north to south and columns are indexed $0, 1, \dots, M - 1$ from west to east. Let (i, j) denote the cell on the i -th row and the j -th column. For each i, j with $0 \leq i \leq N - 1$ and $0 \leq j \leq M - 1$, there is exactly one stand on (i, j) selling cookies shaped like a specific digit from 0 to 9, denoted by $S[i][j]$.

Due to the large flock of tourists, the following regulations have been imposed to make sure everything is under control:

- All visitors must enter the night market from $(0, 0)$ and leave the night market from $(N - 1, M - 1)$.
- If a visitor is on (i, j) , they must buy exactly one $S[i][j]$ -shaped cookie from the stand. After buying the cookie, they must choose to move to $(i + 1, j)$ or $(i, j + 1)$ next. If a cell doesn't exist, they cannot choose that option. In particular, if $(i, j) = (N - 1, M - 1)$, then they must leave the night market after buying the cookie.

Note that due to the regulations above, any visitor will buy exactly $N + M - 1$ cookies before leaving the night market.

Alice and Bob wonder if there is a way for them to visit two different sequences of cells and buy the same sequence of cookie shapes. Two sequences of cells are different if there exists some position such that the cells at this position in each sequence are different.

Given information about the shapes of cookies each stand sells, please write a program that finds two different sequences of cells that satisfy the condition, or reports that no solution exists.

Implementation Details

You should implement the following procedure.

```
std::vector<std::string> find_sequences(int N, int M,
std::vector<std::vector<int>>> S)
```

- N : the number of rows.
- M : the number of columns.
- S : an array of length N where each element is an array of length M .
 - $S[i][j]$ is the shape of the cookie sold by the stand on (i, j) .
- This procedure is called exactly once for each test case.

This procedure should return an array P of length 0 or 2.

- If there is no solution, then P should be an empty array.
- Otherwise, P should contain two strings $P[0], P[1]$ of length $N + M - 2$ representing the two sequences of cells.
 - Both strings must contain exactly $N - 1$ s's and $M - 1$ E's.
 - Define the first sequence of cells $(A[0], B[0]), (A[1], B[1]), \dots, (A[N + M - 2], B[N + M - 2])$ as follows.
 - $(A[0], B[0]) = (0, 0)$.
 - For $0 \leq i \leq N + M - 3$,
 - If $P[0][i]$ is s, then $(A[i + 1], B[i + 1]) = (A[i] + 1, B[i])$.
 - If $P[0][i]$ is E, then $(A[i + 1], B[i + 1]) = (A[i], B[i] + 1)$.
 - Similarly, define the second sequence of cells $(C[0], D[0]), (C[1], D[1]), \dots, (C[N + M - 2], D[N + M - 2])$ using $P[1]$.
 - There must exist an $i, 0 \leq i \leq N + M - 2$ such that $(A[i], B[i]) \neq (C[i], D[i])$.
 - For all $0 \leq i \leq N + M - 2$, $S[A[i]][B[i]] = S[C[i]][D[i]]$ must hold.

Constraints

- $2 \leq N, M \leq 1000$
- $0 \leq S[i][j] \leq 9$ for each i and j such that $0 \leq i < N$ and $0 \leq j < M$.

Subtasks and Scoring

Subtask	Score	Additional Constraints
1	5	$N, M \leq 8$.
2	15	$N = 2$.
3	80	No additional constraints.

For each test case, define W as follows.

- If there is no solution:
 - If an empty array is returned, $W = 1$.
 - If a non-empty array is returned, $W = 0$.
- If there is at least one solution:
 - If the returned array satisfies the conditions specified in the implementation details, then $W = 1$.
 - If the returned array doesn't satisfy the conditions, but does contain two strings consisting of exactly $N - 1$ s's and $M - 1$ e's, then $W = 0.5$.
 - Otherwise, $W = 0$.

For each subtask, let $W_{\min}$ be the smallest W over all test cases and S be the maximum points for this subtask, then you will receive $S \times W_{\min}$ points for this subtask.

Example

Example 1

Consider the following call:

```
find_sequences(2, 4, [[1, 2, 3, 4], [5, 3, 4, 5]])
```

One possible solution is the following pair of sequences:

- $(0,0) \rightarrow (0,1) \rightarrow (0,2) \rightarrow (0,3) \rightarrow (1,3)$
- $(0,0) \rightarrow (0,1) \rightarrow (1,1) \rightarrow (1,2) \rightarrow (1,3)$

The sequence of shapes sold for both sequences is $[1, 2, 3, 4, 5]$.

The procedure can return the following strings that correspond to the sequences above.

```
["EEES", "ESEE"]
```

Example 2

Consider the following call:

```
find_sequences(3, 3, [[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

There is no solution, so the procedure should return an empty array.

Sample Grader

Input format:

```
N M
S[0][0] S[0][1] ... S[0][M-1]
...
S[N-1][0] S[N-1][1] ... S[N-1][M-1]
```

Output format:

```
k
P[0]
P[1]
...
P[k-1]
```

Here, k is the size of the array returned and $P[i]$ is the $(i + 1)$ -th string in the array.